

SoC LLM Workshop: Hands-on with SoCLaaS

Unlock the power of local LLM infrastructure. Build intelligent web clients, CLI copilots, and autonomous agents with zero API cost.

GitHub repo for examples/demos: <https://github.com/lzs/soclaas-demo>

⚡ Supercharging Projects with SoCLaaS

💰 Completely Free of Cost

No paid developer accounts, setup fees, credit card mandates, or school reimbursement claims required. Zero cost barrier to build.

↕ OpenAI Client Compatibility

Exposes standard OpenAI-compatible endpoints. Exposes a familiar API surface, making SoCLaaS **100% drop-in compatible** with standard SDKs, libraries, and tools built for OpenAI.

🛡️ Secure & Private Infrastructure

Hosted locally inside SoC network networks. No prompt leakage, secrets exposure, or usage analytics sent to public clouds.

🎯 The 60-Minute Game Plan

Today is a high-speed, live-guided workshop designed for students to transition from simple chat prompts to backend programmatic integration.

- 1 Issue API keys on central Unix servers
- 2 Connect via REST client curl & Python scripts
- 3 Interact with Open WebUI, work with Codex Copilot, and peek at Agents
- 4 Q&A

Documentation: <https://dochub.comp.nus.edu.sg/cf/guides/soclaas/start>

GitHub Repo for Examples/Demos: <https://github.com/lzs/soclaas-demo>

Get Your API Key in Seconds

API keys are issued through the soclaas-portal command in SoC UNIX servers. You will need to enter your SoC account password.

1. SSH into the Student Unix Cluster using your SoC username

```
ssh <userid>@stu.comp.nus.edu.sg
```

2. Issue or rotate your personalized API credentials

```
soclaas-portal issue
```

Or: Rotate your API key you already had one issued before

```
soclaas-portal issue --rotate
```

3. Source your terminal profile to instantly apply the new variables

```
source ~/.bashrc
```

Success verification:

```
echo $SOCLAAS_BASE_URL
```

```
echo $SOCLAAS_API_KEY
```

```
echo $SOCLAAS_MODEL
```

☰ Model Catalogue & Resource Costs

Model Name	Context Length	Input Cost (μ\$)	Output Cost (μ\$)	Recommended Use Case
llama3.1:8b	65,536 tokens	20,000	30,000	Fast, lightweight experimentation
qwen3.5:9b	65,536 tokens	100,000	150,000	General purpose & multimodal work
gemma4:26b	131,072 tokens	60,000	300,000	Higher-quality general-purpose and multimodal work
qwen3-coder-next	262,144 tokens	110,000	800,000	Coding agents & complex API hooks
qwen3.6:35b	262,144 tokens	140,000	900,000	Advanced reasoning & agent workflows
qwen3.8:27b	262,144 tokens	195,000	900,000	High-quality general reasoning and multimodal tasks
qwen3-vl:32b	80,000 tokens	104,000	416,000	Vision-capable model
ornith:35b	262144 tokens	260,000	900,000	Agentic software engineering and coding workflows

</> Run Your First Call

> REST Request for Model Catalog

Standard HTTP requests using curl.

```
curl -sS "$SOCLAAS_BASE_URL/models" \  
  -H "Authorization: Bearer $SOCLAAS_API_KEY" \  
  -H "Content-Type: application/json" \  
  | jq
```

Example output:

```
{  
  "object": "list",  
  "data": [  
    {  
      "id": "ornith1.5:35b",  
      "object": "model",  
      "created": 0,  
      "owned_by": "soclaas",  
      "context_length": 242144,  
      "context_window": 242144,  
      "max_context_length": 242144,  
      "max_model_len": 242144,  
      "soclaas": {  
        "display_name": "Ornith 1.5 35B A3B",  
        "description": "Agentic software engineering and coding workflows",  
        "capabilities": [  
          "chat"  
        ],  
        "input_microdollars_per_million_tokens": 100000,  
        "output_microdollars_per_million_tokens": 400000,  
        "audio_microdollars_per_minute": 0  
      }  
    },  
    {  
      "id": "qwen3.8:27b",  
      "object": "model",  
      "created": 0,  
      "owned_by": "soclaas",  
      "context_length": 262144,  
      "context_window": 262144,  
      "max_context_length": 262144,  
      "max_model_len": 262144,  
      "soclaas": {  
        "display_name": "Qwen 3.8 27B",  
        "description": "High-quality general reasoning and multimodal tasks",  
        "capabilities": [  
          "chat"  
        ],  
        "input_microdollars_per_million_tokens": 195000,  
        "output_microdollars_per_million_tokens": 900000,  
        "audio_microdollars_per_minute": 0  
      }  
    }  
  ],  
  ...  
}
```

⚙️ LLM API Fundamentals

🔧 Core Variables & Tuning

Prompt vs. System Message

Use system messages to define agent identity, code standards, or behavioral limits; use prompts for immediate requests.

Structured Outputs & JSON Mode

Force models to output strictly valid JSON conforming to schemas. Crucial for connecting API outputs directly to downstream Python logic or frontends.

Tokens & Context Window

Tokens represent parts of words (~0.75 words/token). Keep an eye on context window usage to prevent silent budget inflation.

💰 Understanding Microdollars (μ\$)

SoCLaaS resources are measured in microdollars (μ\$) per million tokens. These are completely abstract units used purely for fair use management across campus networks.

Tokens Processed:	Input (Prompt) + Output (Response)
Key Accounting Rule:	Output tokens cost significantly more!
Cost-per-call:	Sum of prompt + response context

</> Chat Completion Calls

> REST Request to Chat Completion Endpoint

Issue structured completions via standard HTTP requests.

```
curl -sS "$SOCLAAS_BASE_URL/chat/completions" \
-H "Authorization: Bearer $SOCLAAS_API_KEY" \
-H "Content-Type: application/json" \
--data @- <<EOM | jq
{
  "model": "$SOCLAAS_MODEL",
  "messages": [
    {"role": "system", "content": "Be concise."},
    {"role": "user", "content": "Define API gateway."}
  ]
}
EOM

curl -sS -N "$SOCLAAS_BASE_URL/chat/completions" \
-H "Authorization: Bearer $SOCLAAS_API_KEY" \
-H "Content-Type: application/json" \
-d @- <<EOM
{
  "model": "$SOCLAAS_MODEL",
  "messages": [
    {"role": "user", "content": "List 3 embedding uses."}
  ],
  "stream": true
}
EOM
```

◆ OPENAI PYTHON SDK INTEGRATION

Direct drop-in compatibility. Just adjust the base URL and API key fields.

```
#!/usr/bin/env python

from openai import OpenAI
import os

# Instantiate client pointing to SoCLaaS Gateway
client = OpenAI(
    base_url=f"{os.environ['SOCLAAS_BASE_URL']}",
    api_key=os.environ["SOCLAAS_API_KEY"],
)

response = client.chat.completions.create(
    model=os.environ["SOCLAAS_MODEL"], # qwen3.6:35b
    messages=[
        {"role": "user", "content": "Say hello in one sentence."}
    ],
)

print(response.choices[0].message.content)
```

No-Code UI: Open WebUI

Instant Access Portal:

<https://aichat.comp.nus.edu.sg>

Pre-installed and managed by SoC. Just log in and start chatting. This should be considered experimental and for demonstration purposes.

Nice Things You Can Build & Use Inside:

Knowledge Bases

Create your own Google NotebookLM equivalent. Upload slides, papers, or docs to chat locally with custom context.

Model Hot-Swaps

Switch engines mid-chat. Compare reasoning performance from lightweight llama3.1 to advanced coding models instantly.

>_ Codex CLI: Coding in the Terminal

What is Codex?

A fast command-line copilot that generates scripts, executes shell automations, and fixes code bugs directly from your active Unix terminal without launching an IDE.

Configure Codex CLI bindings

```
soclaas-setup-codex
```

Run with the pre-saved profile

```
codex --profile soclaas
```

Build Your Own Chatbot (5 Mins)

Leverage Codex to output a clean, standalone Python CLI loop. Write, test, and adapt custom chatbots programmatically in minutes:

Command to issue inside codex:

Write a self-contained Python chatbot using an OpenAI-compatible API gateway specified in the env SOCLAAS_BASE_URL, SOCLAAS_API_KEY, and SOCLAAS_MODEL. Do not use any external packages.

Output script is ~30 lines:

1. Reads environmental variables
2. Constructs native POST payload
3. Streams or outputs response

Agentic Workflows with OpenClaw

Local & Light Framework

OpenClaw is a highly lightweight, local Python-native agent framework designed for orchestrating autonomous workflows. Rather than just answering queries, OpenClaw agents coordinate **Identities** (system limits), **Tools** (command line scripts, system actions), and **Memory** (persisting state across sessions).

Executable Tool Integration

Give agents the ability to do more than chat. Bind Python structures or command terminal executions to perform complex background automation (e.g. GitHub issue triage, automated repo monitors).

Security, Cost, & Hygiene Check

Strict Key Hygiene

- **Never Hardcode:** Do not save plain-text API keys inside your code scripts. Always retrieve them dynamically via environmental variable reads (`os.environ`).
- **Git Safety:** Ensure `.gitignore` actively isolates environment profiles or configuration folders from entering public repositories.
- **Prompt Safety:** Do not paste passwords, system configurations, or private database tables directly into prompt context fields.

Limits & API Error Responses

401 Unauthorized	Invalid/revoked key. Re-issue via Unix shell.
400 Bad Request	Malformed schema or tool mismatch.
429 Rate Limit	Exceeded 10 RPM or Daily quota. Use backoffs.

Questions?

Official Documentation

Detailed setup guides, prompt standards, and configurations:
<https://dochub.comp.nus.edu.sg/cf/guides/soclaas/start>

Workshop Code Examples

Clone helper templates, python loops, and curl structures:
<https://github.com/lzs/soclaas-demo>

Active API Endpoint

Point your OpenAI compatible clients directly to:
<https://soclaas-api.comp.nus.edu.sg/v1>